

Progetto smf80

Suite C per z/OS di estrazione, filtro e decodifica dei record SMF tipo 80 (eventi RACF) e tipo 81 (inizializzazione RACF) — documento di analisi del codice sorgente

Autore: A. Brezzi (alessandro.brezzi@gmail.com)

Piattaforma: IBM z/OS (mainframe), compilatore x1c , codifica EBCDIC (code page IBM-1140/1144)

Linguaggio: C (C99, con estensioni z/OS: DD: file naming, #pragma filetag)

Composizione cartella: 3 programmi eseguibili, ~25 moduli di libreria, 5 header, 3 makefile, listati di compilazione (.lst)

1. Scopo del progetto

Il progetto implementa strumenti di audit e sicurezza per z/OS: leggono lo stream dei record SMF (System Management Facility) prodotti dal sistema operativo e da RACF (il sottosistema di sicurezza IBM), isolano i record di tipo **80** (eventi di sicurezza RACF: login, violazioni, cambi di autorizzazione, accessi a risorse, ecc.) e tipo **81** (snapshot delle opzioni di inizializzazione RACF, es. SETROPTS), e ne producono estratti o rendicontazioni leggibili in base a criteri di filtro configurabili da un file di parametri esterno.

2. I tre programmi principali

2.1 smf80dmp.c — Dump esadecimale filtrato

- **Input:** DD:UTIPARM (criteri filtro), DD:UTI001 (record SMF, VBS), DD:UTICNTL (PDS con tabelle eventi/sezioni)
- **Output:** DD:UTI002 (stampa esadecimale formattata: header + sezioni rilocabili DTA/DT2)
- Filtra solo i record tipo 80 in base ai criteri, poi produce per ciascuno un dump esadecimale leggibile con intestazione, elenco sezioni presenti e contenuto di ogni sezione rilocabile.
- Al termine stampa statistiche: record totali letti, quanti di tipo 80, quanti estratti, min/max data-ora incontrata, conteggio per singolo evento.

2.2 smf80ext.c — Estrazione record filtrati

- Stessa logica di filtro di smf80dmp , ma l'output (DD:UTI002) è il record SMF originale in formato binario (non un dump), pronto per essere riutilizzato da altri strumenti.
- In più, ogni record di tipo **81** incontrato viene automaticamente decodificato tramite smf81dec() e scritto su DD:UTIDEC .

2.3 smf80tst.c — Versione di sviluppo/test

Contiene una versione precedente/di prototipo, molto simile a smf80ext.c (stessa struttura di filtro), usata presumibilmente durante lo sviluppo come banco di prova prima della versione definitiva.

3. Logica di filtro (comune ai tre programmi)

Il file `UTIPARM` definisce criteri combinati in **AND** tra loro; ogni criterio ha la forma:

```
<nome caratteristica> <operatore> <lista valori> [ + continua su riga successiva ]
```

Caratteristica	Campo record SMF80	Descrizione
EVENT	SMF80EVT	Codice evento RACF
USER	SMF80USR	Utente associato all'evento
RESULT	SMF80EVQ	Event Qualifier (esito evento)
JOBNAM	SMF80JBN	Nome job/address space
REASN	SMF80REA	Motivo del logging (CLAUDIT, UAUDIT, ecc.)
AUTH	SMF80ATH	Autorità usata (SPECIAL, OPERATIONS, ecc.)
RES	Sezione rilocabile 01	Nome risorsa
USRJES	Sezione rilocabile 01	Owner risorsa in classe JESSPOOL
CLASS	Sezione rilocabile 17	Nome classe RACF
PROF	Sezione rilocabile 33	Profilo RACF usato

Operatori ammessi:

Operatore	Significato
=	Uguale a uno dei valori della lista (condizione in OR)
<>	Diverso da tutti i valori della lista (condizione in AND)
ABR	Il valore è abbreviazione di uno della lista

4. Flusso di elaborazione

1. Caricamento tabelle di decodifica da UTICNTL (member SM80EVNT, SM80REL)
2. Lettura e validazione criteri da UTIPARM --> struttura "runparm"
3. Apertura file SMF (UTI001) e file di output
4. Per ogni record SMF letto:
 - se tipo != 80 (e != 81 in smf80ext) -> scarta
 - applica filtri di HEADER (EVENT, USER, RESULT, JOBNAM, REASN, AUTH)
 - carica le sezioni rilocabili (DTA) e rilocabili estese (DT2)
 - applica filtri sulle sezioni (CLASS, RES, USRJES, PROF)
 - se tutti i filtri sono soddisfatti -> scrive record/dump in output
5. Stampa statistiche finali (record letti, estratti, min/max data-ora, per evento)

5. Formato dei record SMF (header di mappatura)

5.1 `smf80fmt.h` — Record SMF tipo 80

- **SMF80HDR**: header fisso del record (data/ora, evento, event qualifier, utente, gruppo, job, terminale, versione RACF, security label...)
- **SMF80DTS**: sezione dati rilocabile "normale" (tipo + lunghezza + dati, max 255 byte), concatenate a partire dall'offset `SMF80REL`
- **SMF80DT2**: sezione dati rilocabile "estesa" (per dati oltre i limiti della sezione normale), a partire dall'offset `SMF80RL2`
- **union `smf80_dta`**: mappatura del contenuto delle sezioni note (es. sezione 01 = nome risorsa, sezione 17 = classe, sezione 33 = profilo)

5.2 `smf81fmt.h` — Record SMF tipo 81

- **SMF81HDR**: header con le opzioni di inizializzazione RACF al momento dell'IPL (nome/volume del database RACF, opzioni SETROPTS, livelli di sicurezza, algoritmo di cifratura password, ecc.)
- **SMF81DTS**, **SMF81S30**, **SMF81S21**: sezioni rilocabili specifiche (es. dataset del DB RACF, classi attive)

6. Moduli di libreria (funzioni di supporto)

Le funzioni di supporto sono compilate in una libreria statica (`myext.a`) linkata dai tre programmi principali.

Modulo	Ruolo
<code>getParm.c</code>	Legge e valida il file di parametri/criteri (UTIPARM), costruisce la struttura <code>runparm</code>
<code>getlrow.c</code>	Legge una riga logica dal file parametri, concatenando le continuazioni terminate con "+"
<code>gettrow.c</code>	Tokenizza la riga logica in caratteristica, operatore e lista di filtri
<code>chkparm.c</code>	Validazione logica dei parametri (verifica esistenza eventi/classi referenziati)
<code>crparm.c</code> / <code>crfilter.c</code>	Costruttori delle liste concatenate di parametri e dei relativi filtri
<code>fltparm.c</code>	Verifica se un valore del record soddisfa i filtri configurati
<code>findevt.c</code>	Ricerca un evento nella lista concatenata per nome o valore
<code>getEvt.c</code>	Carica da PDS (member SM80EVNT) l'elenco degli eventi SMF80 e le descrizioni
<code>getEvq.c</code>	Carica gli Event Qualifier relativi a un evento
<code>getCls.c</code>	Carica le classi RACF definite (estratte da REXX XCDTPROF)
<code>getSez.c</code>	Carica le descrizioni delle sezioni rilocabili (member SM80REL)
<code>createDta.c</code> / <code>creDTA_2.c</code>	Costruttori delle strutture per le sezioni rilocabili normali (DTA) ed estese (DT2)
<code>createStr.c</code>	Costruttore di stringhe a lunghezza variabile (linked list)
<code>smf81dec.c</code>	Decodifica completa dei record SMF 81 (opzioni RACF, richiamato da <code>smf80ext</code>)

hexprt.c	Stampa in formato dump esadecimale di un'area di memoria
fmtDtTm.c	Formattazione di data/ora SMF (incluso formato STCK)
getExt.c	Estrae nome programma ed estensione da <code>__FILE__</code> (per i messaggi di log)
openf.c	Apertura file con parametri z/OS (wrapper su fopen/DD)
makeargv.c	Tokenizzazione generica di una stringa in un array di argomenti
strup.c	Conversione stringa in maiuscolo
trim.c	Funzioni rtrim/ltrim per rimuovere spazi iniziali/finali
box.c	Modulo minimale/stub (nessuna logica implementata rilevata)

7. Header principali

File	Contenuto
smf80ext.h	Strutture dati core (liste concatenate eventi, sezioni, filtri, parametri, runparm) e dichiarazioni extern di tutte le funzioni di libreria
smf80sup.h	Costanti: operatori ammessi, nomi/lunghezze massime dei filtri, tabelle descrittive per REASN, AUTH, DESCR flags
smf80fmt.h	Mapping struct dei record SMF 80 (vedi §5.1)
smf81fmt.h	Mapping struct dei record SMF 81 (vedi §5.2)
strevt.h	Struttura evento minimale, usata dalla variante parstr / smf80tst

8. Build e distribuzione

- Tre makefile in stile mainframe: `smf80dmp.mk`, `smf80ext.mk`, `parstr.mk`
- Compilatore `xlc` con opzione `-Wc,list` (genera i file `.lst` con il listato di compilazione)
- Tutti i moduli di supporto vengono compilati e archiviati nella libreria statica `myext.a`, poi linkata al programma principale
- Il make target di `smf80dmp` copia automaticamente l'eseguibile compilato e il listato verso PDS/PDSE di produzione/debug (`//'ATS.ATX33.LLIB'` e `//'ATS.ATX33.C.DBG'`)
- `smf80dmp 11-2024.zip` : archivio presente nella cartella, verosimilmente uno snapshot/backup della release di novembre 2024

9. Osservazioni tecniche

- **Ambiente EBCDIC/z-OS:** il codice usa `#pragma filetag("IBM-1140")` e imposta il locale `It_IT.IBM-1144`; i typedef come `uint8_t` / `uint32_t` sono ridefiniti manualmente in `smf80ext.h` per compatibilità col compilatore xlc.
- **I/O tramite DD name:** tutti i file sono acceduti tramite nomi logici (`DD:UTIPARM`, `DD:UTI001`, ecc.) tipici di JCL z/OS, non tramite path.
- **Strutture dati:** quasi ovunque liste concatenate scritte a mano (nessuna libreria standard C generica), con pattern costruttore (`crXxx`) + navigazione manuale.

- **Duplicazione di codice:** `smf80dmp.c` e `smf80ext.c` condividono gran parte della logica di lettura/filtro (stesso ciclo, stesse funzioni `str_split` / `findsez` / `check_clock` duplicate in entrambi i file); `smf80tst.c` sembra una copia di lavoro precedente. Un'eventuale factorizzazione del ciclo comune in una funzione di libreria condivisa ridurrebbe la manutenzione parallela dei due sorgenti.
- **Type-safety:** uso estensivo di cast (`uintptr_t`) per navigare le sezioni rilocabili tramite aritmetica su offset, secondo il layout binario IBM dei record SMF (nessun controllo automatico dei limiti, solo verifiche manuali con messaggio di errore su indice fuori range).